

Partition to two-machine weighted completion

Manuscript for expert review
23 September 2026

Abstract

We give an executable search reduction from Partition to weighted completion-time feasibility on two identical machines. Each input number becomes one job whose processing time and weight are twice that number. An exact integral threshold forces the two machine loads to be equal. Any schedule below the threshold yields a Partition witness by reading its machine assignments, while an infeasible target yields the source's no-solution answer. The maps and their encodings have polynomial bit complexity. This is a reconstruction of the construction in Lenstra, Rinnooy Kan, and Brucker (1977, Theorem 3(b)), with an explicit output decoder and finite independent-solver checks.

1. Introduction

Parallel-machine weighted completion time measures both job duration and the cost of waiting. Even two identical machines make its decision form hard. Lenstra, Rinnooy Kan, and Brucker (1977, Theorem 3(b)) proved this by reducing Partition with jobs whose processing times equal their weights. We give the exact integer threshold and both algorithms needed when the problems return witnesses or an explicit no-solution answer.

The main result is a deterministic polynomial-time pair of maps from every positive-integer Partition instance to a legal two-machine instance and from every valid target output back to a valid source output. The same squared-load identity proves feasibility equivalence and makes the decoder independent of job order or idle time. Our scale factor of two keeps the threshold positive and integral for every admitted input. The contribution is an executable reconstruction of the classical rule.

2. Problems and output semantics

Let $a_1, \dots, a_n$ be positive integers, with $n \geq 1$. Partition asks for an index set $I \subseteq \{1, \dots, n\}$ satisfying $\sum_{i \in I} a_i = \frac{\sum_{i=1}^n a_i}{2}$. If none exists, its unique valid output is NO-SOLUTION. Repeated numerical values remain distinct jobs and source indices. The JSON implementation uses zero-based indices.

The target has positive integer processing times p_i and weights w_i , an integer machine count $m \geq 1$, and a positive integer bound K . A valid schedule assigns every job to a machine and a nonnegative start time, without overlap on one machine, such that

$$\sum_{i=1}^n w_i(s_i + p_i) \leq K. \quad (1)$$

Schedules may contain idle time and need not use integer starts. If no such schedule exists, NO-SOLUTION is the valid target output. Each problem therefore has at least one valid output. The executable JSON contract uses integer starts for a schedule witness; when an arbitrary feasible real-start schedule exists, left justification in each machine's job order gives an integral schedule with no larger cost.

3. Construction and exact bound

Define the source total $S = \sum_{i=1}^n a_i$ and square sum $Q = \sum_{i=1}^n a_i^2$. The forward map F makes n jobs on two machines and sets

$$p_i = w_i = 2a_i, \quad m = 2, \quad K = 2Q + S^2. \quad (2)$$

Every target parameter is a positive integer. The recovery map G returns NO-SOLUTION on a valid target NO-SOLUTION output. On any valid schedule, it returns the source indices of the jobs assigned to machine 0. It does not inspect their order or start times.

For a fixed schedule assignment, write L_0 and L_1 for the sums of the original a_i on the two machines. Removing idle time while retaining each machine's order cannot raise any completion time. In the resulting packed schedule, the cost on machine r is

$$\sum_{i \text{ on } r} (2a_i) \left(2 \sum_{j \text{ at or before } i} a_j \right) = 2 \left(L_r^2 + \sum_{i \text{ on } r} a_i^2 \right). \quad (3)$$

Each cross-product occurs once in the left sum; expanding the square on the right counts it twice before the outer factor. Summing Equation 3 over the two machines and using $L_0 + L_1 = S$ gives

$$\text{packed cost} = 2Q + 2(L_0^2 + L_1^2) = K + (L_0 - L_1)^2. \quad (4)$$

4. Correctness

Theorem 1 (search reduction). For every admitted Partition input x and every valid target output y for $F(x)$, the output $G(x, y)$ is valid for x .

Proof. Suppose first that y is a schedule of cost at most K . Removing idle time cannot increase its cost, so Equation 4 implies $(L_0 - L_1)^2 \leq 0$. Thus $L_0 = L_1 = \frac{S}{2}$. The machine-0 indices returned by G form a Partition witness. This argument applies to every valid schedule, including one with idle time or any order.

Conversely, suppose that I is a Partition witness. Place its jobs on machine 0 and all other jobs on machine 1. Run the jobs in any order without idle time from time zero. Both loads are $\frac{S}{2}$, so Equation 4 gives cost exactly K . The target therefore has a valid schedule. The two problems have a feasible witness simultaneously. If y is target NO-SOLUTION, no target schedule exists, so no Partition witness exists and G correctly returns source NO-SOLUTION. $\square$

5. Encoding and running time

Let N be the bit length of the source encoding. The map F performs a linear number of integer additions and multiplications on numbers with $O(N)$ bits. In particular, $S \leq n \max_i a_i$ and $Q \leq n(\max_i a_i)^2$, so K has $O(\log n + \max_i \log a_i)$ bits. The target contains exactly n jobs, two machines, and one bound; its total encoding length and the bit time to compute it are polynomial in N .

The map G parses the source and target output and scans the n machine assignments. Its running time is polynomial in $N + |y|$ and its output has at most n indices. The executable parser and serializer accept arbitrary finite integer sizes, so the JSON interface does not impose CPython's default decimal-digit cap on these bit-complexity claims. No call to a Partition or scheduling solver occurs in either map.

6. Relation to prior work and limits

Theorem 3(b) of Lenstra, Rinnooy Kan, and Brucker (1977) uses $p_i = w_i = a_i$ and an equivalent balanced-load threshold. Scaling each processing time and weight by two multiplies every weighted completion cost by four, giving Equation 2. Their result establishes the hardness attribution. The contribution here is the explicit witness/no-solution map, an all-output proof under the stated search semantics, and a reproducible implementation. The target solver may still require exponential time; the polynomial bounds apply to the reduction maps.

References

J. K. Lenstra, A. H. G. Rinnooy Kan, and P. Brucker. "Complexity of machine scheduling problems." *Annals of Discrete Mathematics* 1 (1977), 343–362. Theorem 3(b), p. 353. Primary PDF.

A. Verification and reproducibility

The finite testing foundation fixes 120 source instances before construction: 16 hand-designed edge cases and 104 seeded random cases, with 30 YES and 90 NO answers. Source sizes are one to eight jobs, with values at most 20. The source oracle enumerates subsets. A Z3 5.1.0.0 scheduling encoding and a separate exhaustive assignment and subset dynamic program independently solve the constructed target instances. Each checker invokes F and G as separate subprocesses and validates target schedules and recovered source outputs directly. Each passed all 120 instances and 150 target outputs, including 30 alternate schedules, after the large-integer repair. These are finite checks; Theorem 1 supplies the general argument.

The initial independent review found that CPython’s default decimal-digit limit rejected a legal large bound during JSON serialization. The corrected CLI disables that limit before parsing and serialization. A retained 8192-bit reproducer and a separate 16384-bit forward and recovery check pass. A focused independent review advanced the repaired result for expert review. No formal proof assistant certificate is claimed.

The repository uses Python 3.12.14, uv 0.12.17, and the locked z3-solver 5.1.0.0 binding. The manuscript was compiled with Typst 0.15.1. From the repository root, the following commands reproduce the corpus gate, oracle self-test, injected target checks, high-bit check, and this PDF. The two echo commands exercise forward construction and output recovery in separate processes.

```
uv sync --locked
uv run research/validate_preparation.py campaigns/partition-parallel-weighted-
completion/work/cases.json
cd campaigns/partition-parallel-weighted-completion/work
uv run check.py --self-test
uv run check.py --candidate algorithm.py
uv run verify.py --candidate algorithm.py
uv run ../reviews/initial/check_large.py
echo '{"numbers":[1,1]}' | uv run algorithm.py
echo '{"source":{"numbers":[1,1]}, "target_solution":{"machines":[0,1], "starts":
[0,0]}}' | uv run algorithm.py --extract
typst compile manuscript.typ manuscript.pdf
```